DEVOPS ASSIGNMENT

Enterprise Student Record & Academic Management Platform

Completed adaptation of the supplied Enterprise Smart Library Management case study

Field	Details
Student Name	Ayush Kiran Jagtap
Program	B.Tech. (AIML)
Assignment	DevOps – Automated CI/CD, Docker & Kubernetes
Application	Enterprise Student Record & Academic Management Platform
Stack	Python 3.11, Flask 3.0, Pytest, Jenkins, Docker, Kubernetes/Minikube
Kubernetes Service	NodePort 30090

1. Assignment Overview

The supplied document presents a production-style Enterprise Smart Library Management Platform and asks students to translate the same engineering and DevOps approach into an Enterprise Student Record & Academic Management Platform. The reference stack is Python 3.11, Flask 3.0, Pytest, Gunicorn, Flake8, Git/GitHub, Jenkins, Docker and Kubernetes/Minikube.

The student version replaces library entities with Students, Faculty, Courses, Enrollments and Grades. The specified business rule is a maximum of 18 credits per semester, and the specified automated-test focus is GPA calculation and enrollment-conflict checks. The student container image is student-academic-platform:v1.0.0 and the Kubernetes service uses NodePort 30090.

The source architecture and technology stack are specified in the supplied guide. filecite turn0file0 L9-L17

The student adaptation requirements are given in the source's comparison table. filecite turn0file0 L769-L792

2. Student Project Requirements

Component	Completed Student Version
Primary Service Domain	Student Records & Academic Grading
Entities	Students, Faculty, Courses, Enrollments, Grades
Business Rule	Maximum 18 credits per semester
Automated Tests	GPA calculation and enrollment conflict checks
Container Image	student-academic-platform:v1.0.0
Kubernetes Service	NodePort 30090

3. GitHub Repository Structure

```
student-academic-platform/
├── app/
│   ├── __init__.py
│   ├── routes/api.py
│   ├── services/academic_service.py
│   └── models/
├── database/
│   └── db.py
├── tests/
│   ├── test_student_service.py
│   ├── test_enrollment.py
│   ├── test_gpa.py
│   └── test_api_endpoints.py
├── k8s/
│   ├── deployment.yaml
│   └── service.yaml
├── Jenkinsfile
├── Dockerfile
├── .dockerignore
├── .gitignore
├── requirements.txt
├── run.py
└── README.md
```

The source requires app/, database/, tests/, k8s/, Jenkinsfile, Dockerfile and dependency/configuration files. It also defines separate feature branches for foundation, core application, automated tests, Jenkins CI, Dockerization and Kubernetes, with main as the stable branch. filecite turn0file0 L19-L22 filecite turn0file0 L60-L73

4. Topic 1 – Environment Setup

```
sudo apt-get update && sudo apt-get upgrade -y
sudo apt-get install -y python3 python3-pip python3-venv git curl wget jq

git --version
python3 --version
pip3 --version

git config --global user.name "Ayush Kiran Jagtap"
git config --global user.email "your-email@example.com"
git config --global init.defaultBranch main

mkdir -p student-academic-platform
cd student-academic-platform
git init
git status
```

The reference guide uses Ubuntu 22.04 LTS, WSL2 or macOS and gives the same prerequisite installation flow. filecite turn0file0 L74-L99

4.1 .gitignore

```
__pycache__/
*.py[cod]
venv/
.venv/
.pytest_cache/
.coverage
htmlcov/
reports/
*.xml
*.log
.DS_Store
```

5. Topic 2 – Application Dependencies

```
Flask==3.0.3
Werkzeug==3.0.3
pytest==8.2.1
pytest-cov==5.0.0
gunicorn==22.0.0
flake8==7.0.0
requests==2.31.0
```

6. Academic Database

```
import threading

class Database:
    _instance = None
    _lock = threading.Lock()
```

```
    def __new__(cls):
        with cls._lock:
            if cls._instance is None:
                cls._instance = super().__new__(cls)
                cls._instance._init_db()
            return cls._instance

    def _init_db(self):
        self.students = {}
        self.faculty = {}
        self.courses = {}
        self.enrollments = []
        self.grades = []
        self._seed()

    def _seed(self):
        self.students["S101"] = {
            "student_id": "S101",
            "name": "Alice Johnson",
            "email": "alice@university.edu"
        }
        self.faculty["F101"] = {
            "faculty_id": "F101",
            "name": "Dr. Robert Smith",
            "email": "robert.smith@university.edu"
        }
        self.courses["C101"] = {
            "course_id": "C101", "name": "Python Programming", "credits": 4
        }
        self.courses["C102"] = {
            "course_id": "C102", "name": "Data Structures", "credits": 4
        }

    def reset(self):
        self.students.clear()
        self.faculty.clear()
        self.courses.clear()
        self.enrollments.clear()
        self.grades.clear()
        self._seed()

db = Database()
```

7. Academic Business Logic

```
from database.db import db

GRADE_POINTS = {
    "A+": 10.0, "A": 9.0, "B+": 8.0, "B": 7.0,
    "C+": 6.0, "C": 5.0, "D": 4.0, "F": 0.0
}

class AcademicService:

    @staticmethod
    def enroll_student(student_id, course_id, semester):
        if student_id not in db.students:
            raise ValueError("Student not found.")
        if course_id not in db.courses:
            raise ValueError("Course not found.")
```

```
        for e in db.enrollments:
            if e["student_id"] == student_id and                e["course_id"] == course_id and
e["semester"] == semester:
                raise ValueError("Enrollment conflict: student already enrolled.")

        credits = sum(
            db.courses[e["course_id"]]["credits"]
            for e in db.enrollments
            if e["student_id"] == student_id and e["semester"] == semester
        )

        if credits + db.courses[course_id]["credits"] > 18:
            raise ValueError("Credit limit exceeded. Maximum is 18 credits.")

        record = {
            "student_id": student_id,
            "course_id": course_id,
            "semester": semester
        }
        db.enrollments.append(record)
        return record

    @staticmethod
    def add_grade(student_id, course_id, semester, grade):
        if grade not in GRADE_POINTS:
            raise ValueError("Invalid grade.")

        enrolled = any(
            e["student_id"] == student_id and
            e["course_id"] == course_id and
            e["semester"] == semester
            for e in db.enrollments
        )
        if not enrolled:
            raise ValueError("Student is not enrolled in this course.")

        record = {
            "student_id": student_id,
            "course_id": course_id,
            "semester": semester,
            "grade": grade,
            "points": GRADE_POINTS[grade]
        }
        db.grades.append(record)
        return record

    @staticmethod
    def calculate_gpa(student_id, semester):
        records = [
            g for g in db.grades
            if g["student_id"] == student_id and g["semester"] == semester
        ]
        if not records:
            raise ValueError("No grades available.")

        total_points = 0
        total_credits = 0

        for r in records:
            credits = db.courses[r["course_id"]]["credits"]
```

```
            total_points += r["points"] * credits
            total_credits += credits

        return round(total_points / total_credits, 2)
```

8. REST API

```
from flask import Blueprint, request, jsonify
from app.services.academic_service import AcademicService

api_bp = Blueprint("api", __name__)

@api_bp.route("/health", methods=["GET"])
def health():
    return jsonify({
        "status": "UP",
        "service": "Enterprise Student Academic Platform",
        "version": "1.0.0"
    }), 200

@api_bp.route("/enroll", methods=["POST"])
def enroll():
    data = request.get_json() or {}
    try:
        result = AcademicService.enroll_student(
            data.get("student_id"),
            data.get("course_id"),
            data.get("semester")
        )
        return jsonify({"message": "Enrollment successful",
                        "enrollment": result}), 200
    except ValueError as e:
        return jsonify({"error": str(e)}), 400

@api_bp.route("/grades", methods=["POST"])
def grade():
    data = request.get_json() or {}
    try:
        result = AcademicService.add_grade(
            data.get("student_id"),
            data.get("course_id"),
            data.get("semester"),
            data.get("grade")
        )
        return jsonify({"message": "Grade added successfully",
                        "grade": result}), 201
    except ValueError as e:
        return jsonify({"error": str(e)}), 400

@api_bp.route("/students/<student_id>/gpa/<semester>")
def gpa(student_id, semester):
    try:
        value = AcademicService.calculate_gpa(student_id, semester)
        return jsonify({"student_id": student_id,
                        "semester": semester, "gpa": value}), 200
    except ValueError as e:
        return jsonify({"error": str(e)}), 400
```

9. Flask Application Factory

```
# app/__init__.py
from flask import Flask

def create_app():
    app = Flask(__name__)
    from app.routes.api import api_bp
    app.register_blueprint(api_bp, url_prefix="/api")
    return app

# run.py
import os
from app import create_app

app = create_app()

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 5000))
    app.run(host="0.0.0.0", port=port, debug=False)
```

10. Topic 3 – Automated Testing

The source guide uses Pytest for service and API testing and runs the suite with coverage plus JUnit XML output. The student tests below directly cover the assignment's GPA and enrollment requirements. filecite turn0file0 L396-L405

```
import pytest
from database.db import db
from app.services.academic_service import AcademicService

@pytest.fixture(autouse=True)
def reset_db():
    db.reset()

def test_enrollment_success():
    result = AcademicService.enroll_student("S101", "C101", "2026-S1")
    assert result["student_id"] == "S101"

def test_duplicate_enrollment_fails():
    AcademicService.enroll_student("S101", "C101", "2026-S1")
    with pytest.raises(ValueError, match="Enrollment conflict"):
        AcademicService.enroll_student("S101", "C101", "2026-S1")

def test_credit_limit():
    db.courses["C103"] = {"course_id":"C103","name":"AI","credits":15}
    db.courses["C104"] = {"course_id":"C104","name":"ML","credits":4}
    AcademicService.enroll_student("S101", "C103", "2026-S1")
    with pytest.raises(ValueError, match="Credit limit exceeded"):
        AcademicService.enroll_student("S101", "C104", "2026-S1")

def test_gpa_calculation():
    AcademicService.enroll_student("S101", "C101", "2026-S1")
    AcademicService.enroll_student("S101", "C102", "2026-S1")
    AcademicService.add_grade("S101", "C101", "2026-S1", "A")
    AcademicService.add_grade("S101", "C102", "2026-S1", "B")
    assert AcademicService.calculate_gpa("S101", "2026-S1") == 8.0
```

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
mkdir -p reports
pytest -v --cov=app --cov-report=term-missing        --junitxml=reports/test-results.xml tests/
```

11. Topic 4 – Jenkins CI/CD

The reference Jenkins pipeline contains checkout, environment setup, linting, tests, Docker build and Kubernetes deployment stages. The student pipeline below follows that structure. filecite turn0file0 L518-L520

```
pipeline {
    agent any

    environment {
        PYTHONUNBUFFERED = '1'
        DOCKER_IMAGE_NAME = 'student-academic-platform'
        IMAGE_TAG = "v1.${BUILD_NUMBER}"
    }

    stages {
        stage('SCM Checkout') {
            steps { checkout scm }
        }

        stage('Environment Setup') {
            steps {
                sh '''
                    python3 -m venv venv
                    . venv/bin/activate
                    pip install --upgrade pip
                    pip install -r requirements.txt
                '''
            }
        }

        stage('Linting') {
            steps {
                sh '''
                    . venv/bin/activate
                    flake8 app/ database/ --count                    --select=E9,F63,F7,F82
--show-source --statistics
                '''
            }
        }

        stage('Automated Tests') {
            steps {
                sh '''
                    . venv/bin/activate
                    mkdir -p reports
                    pytest -v --cov=app                    --junitxml=reports/test-results.xml tests/
                '''
            }
        }

        stage('Docker Build') {
            steps {
                sh '''
```

```
                    docker build -t ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} .
                    docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG}
${DOCKER_IMAGE_NAME}:latest
                '''
            }
        }

        stage('Kubernetes Deployment') {
            steps {
                sh '''
                    kubectl apply -f k8s/deployment.yaml
                    kubectl apply -f k8s/service.yaml
                    kubectl rollout status deployment/student-deployment                    --
timeout=60s
                '''
            }
        }
    }

    post {
        always {
            junit allowEmptyResults: true,
                  testResults: 'reports/test-results.xml'
            cleanWs deleteDirs: true, notFailBuild: true
        }
        success {
            echo 'CI/CD pipeline completed successfully.'
        }
        failure {
            echo 'Build failed. Check Jenkins logs.'
        }
    }
}
```

12. Jenkins Installation

```
sudo apt-get update
sudo apt-get install -y openjdk-17-jre
java -version

curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee
/usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable
binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null

sudo apt-get update
sudo apt-get install -y jenkins
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

13. Topic 5 – Dockerfile

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1     PYTHONUNBUFFERED=1     PORT=5000
```

```
WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends     curl && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m -u 1000 appuser &&     chown -R appuser:appuser /app
USER appuser

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=5s     --start-period=5s --retries=3     CMD curl -f
http://localhost:5000/api/health || exit 1

CMD ["python", "run.py"]
```

This preserves the reference guide's Python 3.11-slim base image, non-root execution and healthcheck approach. filecite turn0file0 L625-L660

13.1 Docker Commands

```
docker build -t student-academic-platform:v1.0.0 .
docker images

docker run -d -p 5000:5000   --name academic-container   student-academic-platform:v1.0.0

docker ps
docker logs -f academic-container
curl http://localhost:5000/api/health

docker stop academic-container
docker rm academic-container
```

14. Topic 6 – Kubernetes Deployment

14.1 deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: student-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: student-platform
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: student-platform
    spec:
```

```
      containers:
        - name: student-app
          image: student-academic-platform:v1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          resources:
            requests:
              memory: "128Mi"
              cpu: "100m"
            limits:
              memory: "256Mi"
              cpu: "250m"
          livenessProbe:
            httpGet:
              path: /api/health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 20
          readinessProbe:
            httpGet:
              path: /api/health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
```

14.2 service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: student-service
spec:
  type: NodePort
  selector:
    app: student-platform
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
      nodePort: 30090
```

14.3 Minikube Commands

```
minikube start --driver=docker
eval $(minikube docker-env)

docker build -t student-academic-platform:v1.0.0 .

kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml

kubectl get pods -l app=student-platform -o wide
kubectl get svc student-service

kubectl delete pod $(kubectl get pods -l app=student-platform -o jsonpath='{.items[0].metadata.name}')

kubectl get pods -w

minikube service student-service --url
```

The source guide demonstrates the same Minikube deployment, replica verification, pod deletion/self-healing and service access sequence. filecite turn0file0 L748-L768

15. End-to-End CI/CD Flow

- Developer pushes code to GitHub.
- Jenkins checks out the repository.
- Dependencies are installed in a Python virtual environment.
- Flake8 performs the syntax/static check.
- Pytest runs unit and integration tests.
- JUnit XML results are published in Jenkins.
- Docker builds the student application image.
- Kubernetes manifests are applied to Minikube.
- Three replicas are created and health probes check them.
- NodePort 30090 exposes the application.
- If a pod is deleted, Kubernetes automatically recreates it.

16. API Test Examples

```
curl http://localhost:5000/api/health

curl -X POST http://localhost:5000/api/enroll \
-H "Content-Type: application/json" \
-d '{"student_id":"S101","course_id":"C101","semester":"2026-S1"}'

curl -X POST http://localhost:5000/api/grades \
-H "Content-Type: application/json" \
-d '{"student_id":"S101","course_id":"C101","semester":"2026-S1","grade":"A"}'

curl http://localhost:5000/api/students/S101/gpa/2026-S1
```

17. Expected Verification

Test / Check	Expected Result
Health API	HTTP 200, status UP
Valid enrollment	Successful enrollment
Duplicate enrollment	Rejected with conflict error
More than 18 credits	Rejected with credit-limit error
Grade entry	Successful when student is enrolled
GPA	Weighted GPA calculated correctly
Docker	Container runs on port 5000
Kubernetes	3 healthy replicas
Service	NodePort 30090
Self-healing	Deleted pod is recreated automatically

18. Git Commit / Branch Evidence

```
git checkout -b feature/foundation
git add . && git commit -m "Add project foundation"

git checkout -b feature/core-app
```

```
git add . && git commit -m "Implement academic core application"

git checkout -b feature/automated-tests
git add . && git commit -m "Add automated tests"

git checkout -b feature/jenkins-ci
git add Jenkinsfile && git commit -m "Add Jenkins CI pipeline"

git checkout -b feature/dockerization
git add Dockerfile .dockerignore && git commit -m "Add Docker support"

git checkout -b feature/kubernetes-orchestration
git add k8s/ && git commit -m "Add Kubernetes deployment"
```

19. Submission Checklist

No.	Required Deliverable	What to Submit
1	GitHub Repository	app/, database/, tests/, k8s/, Dockerfile, Jenkinsfile
2	Branch/PR Evidence	Git history or screenshots of required branch progression
3	Jenkins Evidence	Successful pipeline and JUnit test report screenshot
4	Docker Evidence	docker images and docker ps output
5	Minikube Evidence	3 healthy pods plus self-healing after pod deletion
6	Technical Report	This report with setup, implementation and DevOps explanation

These six deliverables match the source submission checklist. filecite turn0file0 L794-L803

20. Viva Questions – Short Answers

What is the project? A Flask-based platform for student records, course enrollment and academic grades.

Why Jenkins? To automate build, test and deployment steps.

Why Docker? To package the application and dependencies into a portable container.

Why Kubernetes? To manage replicas, health checks, rolling updates and self-healing.

What is the credit rule? A student can enroll for a maximum of 18 credits in one semester.

What is tested? GPA calculation, duplicate enrollment and credit-limit validation.

Which port is used? Kubernetes NodePort 30090; the Flask container listens on port 5000.

How is self-healing demonstrated? Delete one pod and observe Kubernetes automatically create a replacement.

21. Conclusion

The completed student project translates the supplied library case study into an academic-management domain while keeping the same DevOps engineering pattern. It provides a Flask application, modular service logic, automated

Pytest validation, Jenkins CI/CD, Docker containerization and Kubernetes/Minikube orchestration. The student-specific requirements are implemented through the 18-credit semester limit, enrollment-conflict validation and GPA calculation, with the required image tag and NodePort.